实习项目 · 第七周技术汇报

本地长上下文 Agent 部署与推理优化

从个人 Spark 到公司级 DeepSeek 服务

01
SPARK
个人方案

02
DEEPSEEK
公司方案

03
A100
推理实现

04
AI INFRA
系统方法

LONG-CONTEXT

1M

容量不是全部



容量 / 响应 / 并发 / 成本

2026.10.15

选定模型之后，还要回答四个部署问题

四部分按“个人方案 → 公司方案 → 工程案例 → 分析方法”展开。

01

装得下?

上下文容量

权重、 KV Cache 与
每位用户的可用空间

02

等多久?

响应与生成

首次 prefill 、缓存命中
以及 decode 速度

03

能共享?

并发与调度

同时生成的请求
与需要保留的会话

04

花多少?

部署与成本

硬件交付、实施
与真实任务的经济性

Spark 个人方案 | DeepSeek 公司方案 | A100 优化案例 | AI Infra Book

依据：讲稿「开场」及四部分结构。

上下文是整个 Agent 请求的工作空间

不仅是上传的资料，也包括系统指令、历史对话、工具结果和本次输出。

系统指令 | **资料与对话** | **代码与工具结果** | **本次输出**

总上下文 = 输入 + 输出

131K	262K	512K	1M
基础要求	基础要求	长上下文档位	长上下文档位

每档预留 8,192 token 用于输出

512K = 524,288 · 1M = 1,000,000

依据：讲稿「开场」。示意图不按各组成部分的实际 token 比例绘制。

模型在 Spark，工具在员工电脑

OpenCode 保留本地工作目录、身份和 NAS 权限； Spark 的内存优先留给模型与 KV Cache 。

员工电脑

OpenCode

文件读写 / Python / Excel

本地身份与 NAS 访问权限

指令 / 资料 / 工具结果

答案 / 工具调用

DGX Spark · 128GB

Qwen3.8-27B

共享模型推理服务

模型权重 + 各会话 KV Cache

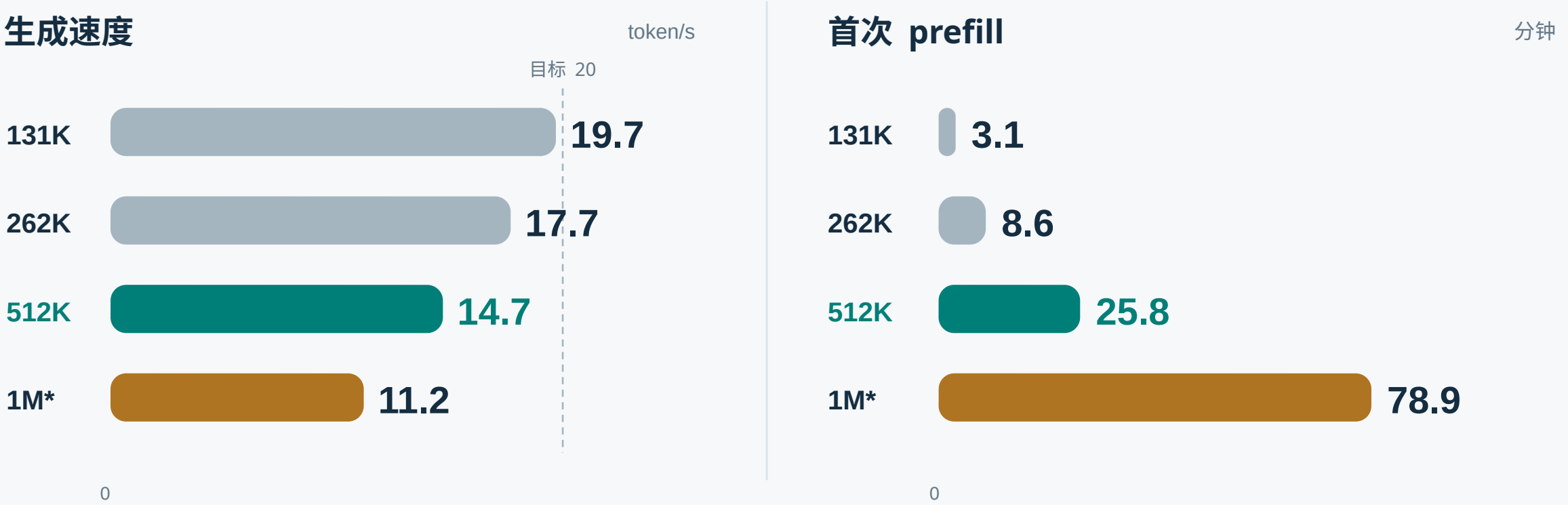
1 读两期财报 → 2 统一科目与单位 → 3 运行脚本计算 → 4 回到原文核对 → 5 **Excel / 图表 / 说明**

模型可以共享；每个人的文件、账号和会话仍保持独立。

来源： speaker-notes.md 一.1； Qwen 模型卡。个人方案暂定 Qwen3.8-27B；模型与工具分开部署。

上下文拉长，首次等待比生成速度更快恶化

单请求接近用满各档上下文，均预留 8K 输出； FP8 权重 + FP8 KV Cache + MTP3 。



512K ≈ 15 token/s ，仍低于单人 20 token/s 的目标

来源：外部 Spark 测试汇总与讲稿一 .2–3 。四档为估算； 1M 为外推，完整样本见附录 A （ 28 页）。

先缓存长材料，再处理本轮新增的工具结果

两种请求的总上下文都接近 512K；旧输入的 KV Cache 与线性注意力状态已保留在 GPU。

估算 · 缓存命中

首次读完整输入：约 25.8 分钟

已有输入：直接复用缓存 → 新增 1K 工具结果 → ≈ 5 秒

新增 prefill

已有输入：直接复用缓存 → 新增 8K 工具结果 → ≈ 40 秒

新增 prefill

缓存命中省去重复 prefill，但 decode 仍要读取上下文对应的 KV。

实际首 token 等待 = 请求排队 + 新增 prefill + 第一步 decode。

来源：讲稿一 .2–3。 5 秒与 40 秒仅为新增 prefill 估算，假设旧 KV 与线性注意力状态已在 GPU。

容量看会话总和，速度看谁在同时运行

Spark 的 CPU / GPU 共用 128GB； CPU 缓存副本不扩容，闲置长会话可保存到 NVMe 。

1.81M token · 共享 KV 池

三人均分



一人使用 1M ，其余两人共用约 810K



三路实测，每路输入 100K

每路输出 4,096 token

17.5 token/s

共同 decode 窗口 · 每路平均

12.5 token/s

各自首 token 至结束 · 含干扰等待

FP8 KV 预留 10% 余量：三人仍可各配置 512K

来源：外部 Spark 测试汇总。左侧为容量；右侧为三路各 100K 输入、4,096 输出的实测。

一台 Spark：个人部署与三人共享的投入

同一台模型服务器；实施成本主要来自首次配置、工具接入与功能检查。

个人使用

6.8–8.2

万元 · 硬件与实施合计

硬件 5.8–6.2 万 + 实施 1–2 万

三人共享

7.8–9.9

万元 · 硬件与实施合计

同一台 Spark，增加多用户接入工时

硬件预算： $6,950 × 假设汇率 7.00，再计税费、运输与支持准备。

自行部署时，实施费用对应团队工时；此处是预算范围，不是境内成交报价。

依据：讲稿 一 .5；原版 PDF 附录 D。预算观察日期 2026-10-12；人民币万元。

共享模型，隔离每个员工或项目的任务环境

目标：同时运行 20 个任务，每个最多 1M 上下文；另设 131K、 262K、 512K 档位。

员工 / 项目独立

Hermes 会话

独立文件目录与执行环境

Skills： 步骤、模板与任务记录

输入与结果

答案与调用

共享模型服务

DeepSeek V4.1-Flash

GPU

模型权重 + 会话 KV

主机内存

≈ 203GB Engram

MCP

业务工具

员工电脑或 CPU 服务器执行

公司知识库

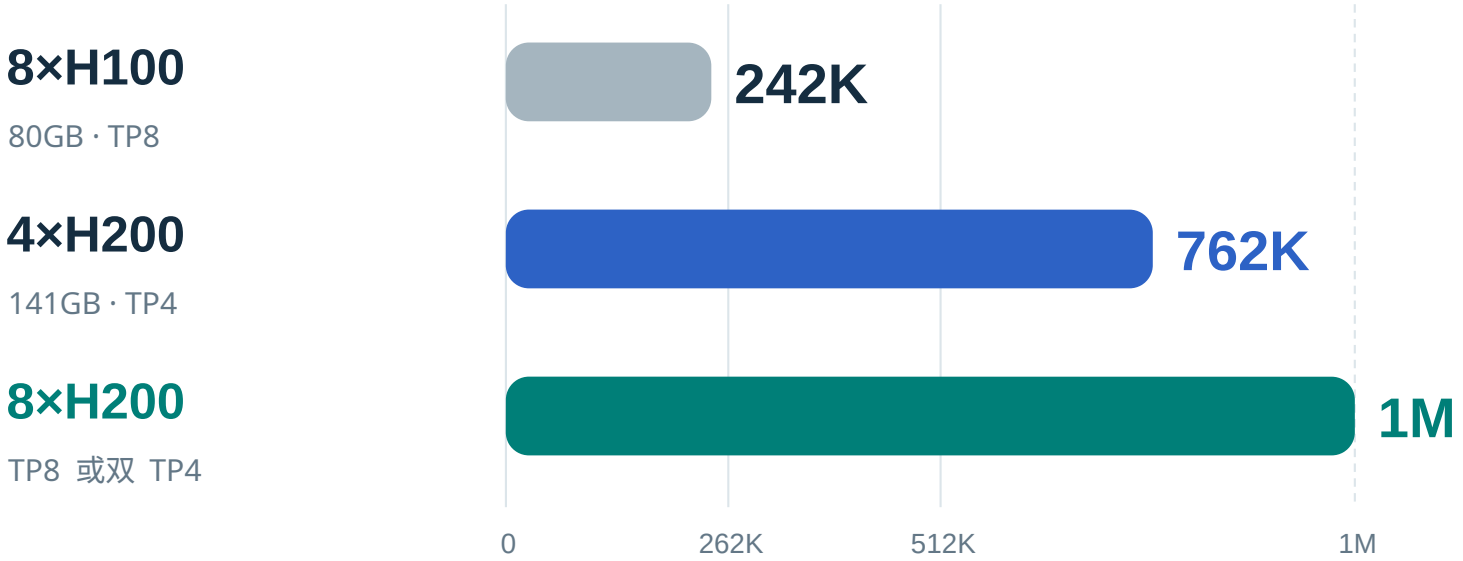
按身份检索 NAS，并返回原文出处

来源：讲稿二 .1–2 。模型权重、 Engram 与会话 KV 分开计账；公司知识库简版见附录 F （33 页）。

八卡 H200 的缓存容量可容纳 20 × 1M 会话

按公开启动日志的可用 KV 预算估算， 20 个会话均分，并预留 10% 缓存容量。

每会话可用上下文



当前 H100 / H200 实现

1.83

GB / 卡 / 1M 会话

20 个 1M 会话

≈ 36.5GB / 卡

H100 的约 242K 低于 262K；四卡 H200 可容纳 20 个 512K。

图中八卡 H200 按 1M 档位封顶；局部状态、草稿模型与运行缓冲另占显存。

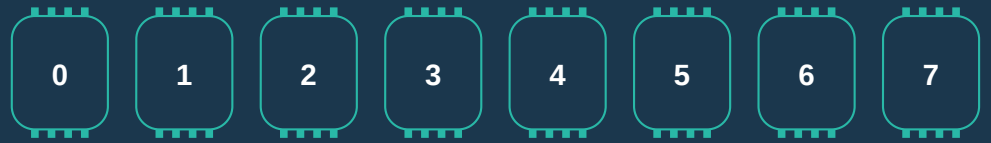
来源：公开启动池汇总与讲稿二 .2–3 。仅估算 GPU 缓存容量；其他状态另计，详见附录 B （ 29 页）。

同一台八卡服务器，比较 TP8 与双 TP4

硬件购置费相同，差别在于请求分组、队列、模型副本与 KV Cache 的分配。

一个 TP8 实例

20 人 · 一个共享队列



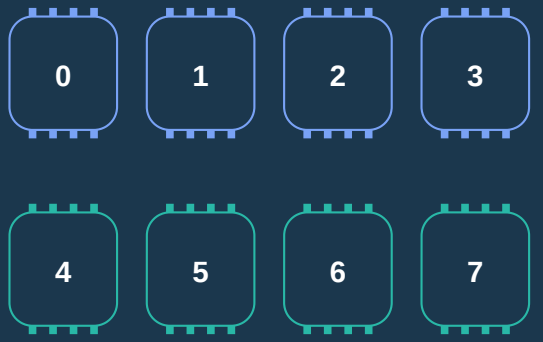
8 张卡协同处理同一实例的请求

1 份 Engram · 主机内存约 203GB

两个 TP4 实例

实例 A · 10 人

实例 B · 10 人



两组队列，各自保留 KV Cache

独立双服务： 2 份 Engram ，约 406GB

最终选择要看： 20 路的每路速度，以及长上下文工具返回后的等待。

两份预算对应独立服务； vLLM 0.31 同机 DP 可共享 Engram ，需按最终部署核对。

20 路生成，不等于只保留 20 份上下文

等待 Excel 计算或检索的任务，如果要直接继续推理，也需要保留各自缓存。

20

正在 decode



+

20

正在等待工具



=

40

需要保留的上下文

按缓存容量分别预算

GPU 优先运行任务，闲置状态分层保存

20 个生成 + 20 个等待 = 40 份会话状态

20 路 × 20 token/s = 400 token/s

这是系统持续输出目标，不是已验证结果

来源：讲稿 二 .4–5； handoff 三、五。分层方案见下一页；缓存取回算例见第 25 页。

GPU 运行当前任务，内存和 SSD 保留其他会话

公司服务器： GPU 显存与主机 DRAM 独立；工具执行期间， GPU 空间可供其他请求使用。



1. **模型返回工具调用**
   本轮模型请求结束。
2. **工具执行， GPU 服务其他任务**
   状态保存在 CPU 内存或 SSD 。
3. **工具结果返回，查找并读取缓存**
   把已有状态从内存复制回 GPU 。
4. **处理新增输入，再继续生成**
   复用旧前缀，生成本轮的新回答。

同时运行的任务数看 GPU ；可保留的会话数看整套缓存系统。

实现路线 1：vLLM 原生 KV 卸载

实现路线 2：vLLM + LMCache MP

优先保留仍在进行、值得复用的会话

来源： AI Infra Book 8.3 、 9.5 ； vLLM KV 卸载指南； LMCache MP 。缓存指推理状态，不是完整答案缓存。

公开回放很快，但还不能替代 20 路满 1M 的实测

容量通过筛选后，需要用真实长上下文与工具返回负载比较两种布局。

公开参照 · H200 TP8

32 个回放会话

1,052

token/s · 系统总输出

2.63

秒 · p95 首 token 等待

输入约 140K–157K，前缀命中约 95%

DSpark5；平均接受长度预设为 3.51

待验证

20 路 × 1M

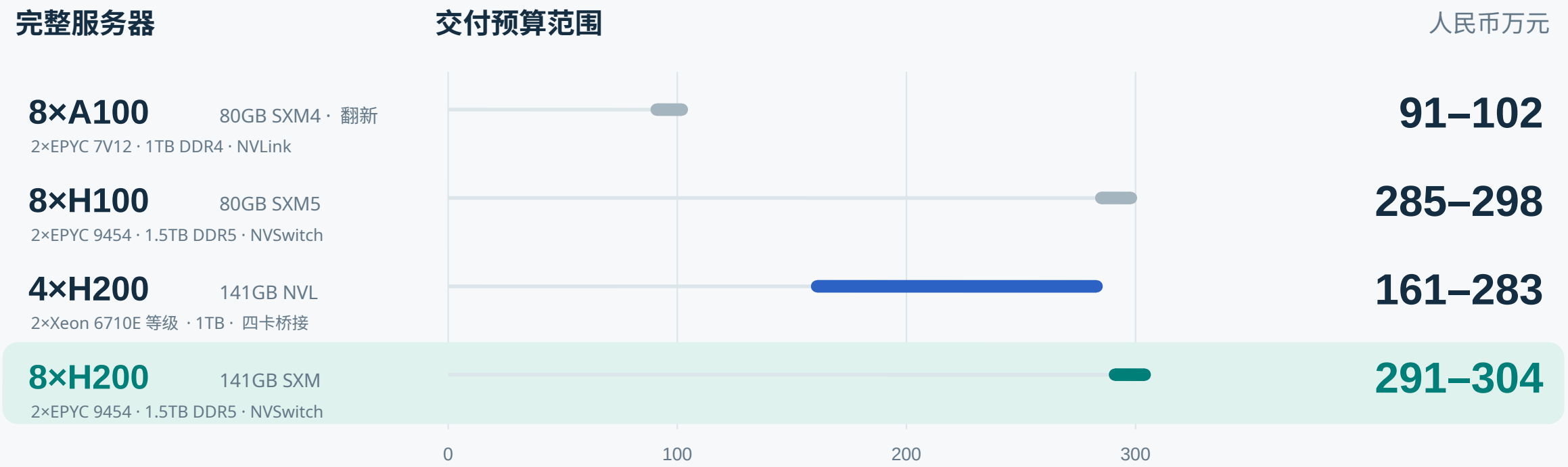
布局	**TP8 vs 双 TP4**
上下文	**512K / 1M**
工具返回	**新增 1K / 8K**

比较每路 decode + 工具返回后的 TTFT

来源： AgentX 公开回放汇总与讲稿二 .5 。高前缀命中、预设草稿接受行为；详见附录 C （30 页）。

公司候选：八卡 H200；已有 A100 按复用成本评估

整机交付预算； CPU 、主机内存与机内互联随下方列示，完整配置见附录 D （第 31 页）。



同一供应商的八卡 H100 / H200 挂牌价仅差约 2%

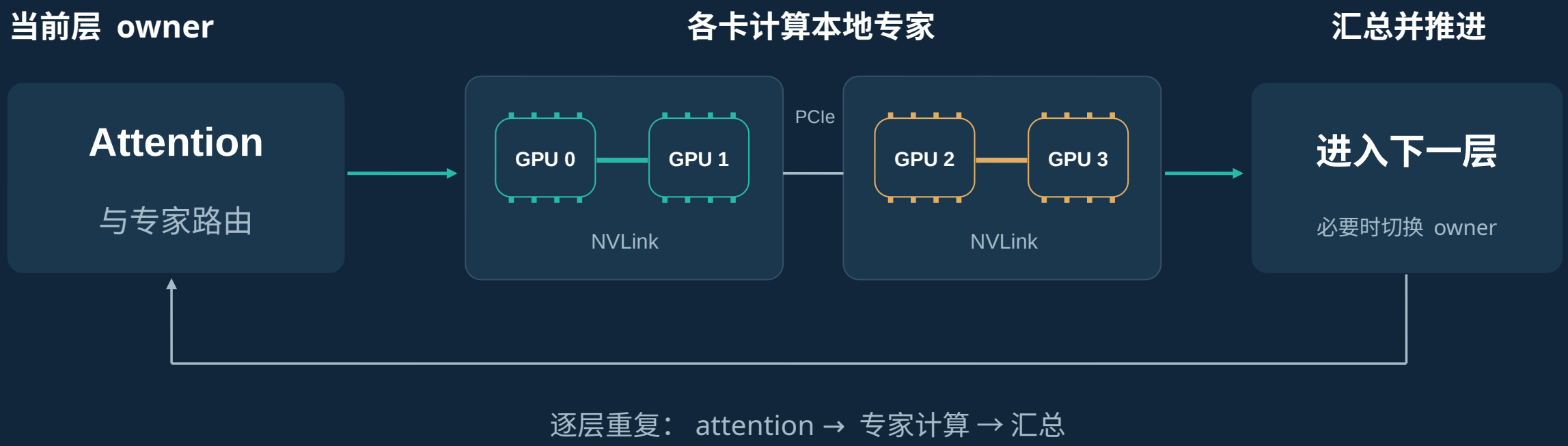
集中工具执行另加 12–26 万

四卡 H200 为 NVL ，不能直接套用其他硬件的性能；采购仍需核对 SKU 、交付路线与许可。

来源： handoff 整机配置与报价入口； 2026-10-12 预算，假设 USD/CNY=7.00 。详见第 31 页。

shi3z 的价值：把新模型适配到四卡 A100

作者文章标题：“A100 从 33 → 673 token/s，超过官方 API”。从批量生成转向 Coding Agent。



最初先优化批量 decode；接入 Coding Agent 后补长输入与多轮缓存。

来源： shi3z 9/12 、 9/17 文章公开部分；固定源码 580568e 。硬件： 4×A100 PCIe ，两对 NVLink 。

三项工程经验：先定位瓶颈，再整合优化

特色是 A100 适配、按硬件拓扑安排通信，以及 CUDA Graph 与会话缓存的协调。

01

先减少逐 token 搬运

先调整专家权重的存放位置，
减少 PCIe 传输；
再优化矩阵乘法内核。

02

按 NVLink 卡对通信

跨卡对先走一次 PCIe ，
再在卡对内部经 NVLink 分发；
返回结果先在卡对内合并。

03

协调 Graph 与缓存

CUDA Graph 依赖固定地址。
缓存扩容、恢复 KV / 索引 /
压缩器状态需要一起设计。

模型已有	原生 FP4 / FP8 、共享 KV 、 CED 、 DSpark	**引擎已有**	并行、 Graph 、融合、缓存与分块

本项目的贡献在实现与整合，不能把模型原生能力全部算成引擎创新。

依据：讲稿 三 .2 、三 .4 。模型能力、通用方法与本项目实现分开归属。

不同硬件与负载，不能直接排出引擎快慢

以下保留每条结果的硬件和测试条件；单位均为 token/s 。

实现与硬件	单路输出	系统总输出	负载与统计条件
shi3z 定制引擎 4×A100 80GB PCIe	**50–53 / 82–93** 代码请求 / MTP 实验	— 讲稿未列同口径总吞吐	**短输入** 两个不同实验
modified vLLM 8×A100 SXM · 历史 TP8	**110–140** DSpark5 decode	**1,040–1,379** 32 并发	**短输入** 输出 1,024 token
modified vLLM 8×A800 SXM · 优化配置	**≈ 115** 单客户端持续输出	**≈ 3,040** 128 并发	**输入 78–90 token** 输出 1,024 token

目前没有证据证明 shi3z 引擎比 modified vLLM 更快。

来源： shi3z 、 keplerzip 及 Tokha233 公开记录，沿用 10/14 归档。每行保留各自硬件与负载。

modified vLLM 同机消融：每项改动带来多少收益

Tokha233 · 8×A800 · 128 并发；实际输入 78–90 token 、输出 1,024 token ； 每档两次。

两次平均总输出吞吐

基线含 DSpark 、 Marlin 与 CUDA Graph

基线： Marlin dense

NCCL EP8

2,842.48

大 batch 稠密层

改走 BF16 算子

2,926.08

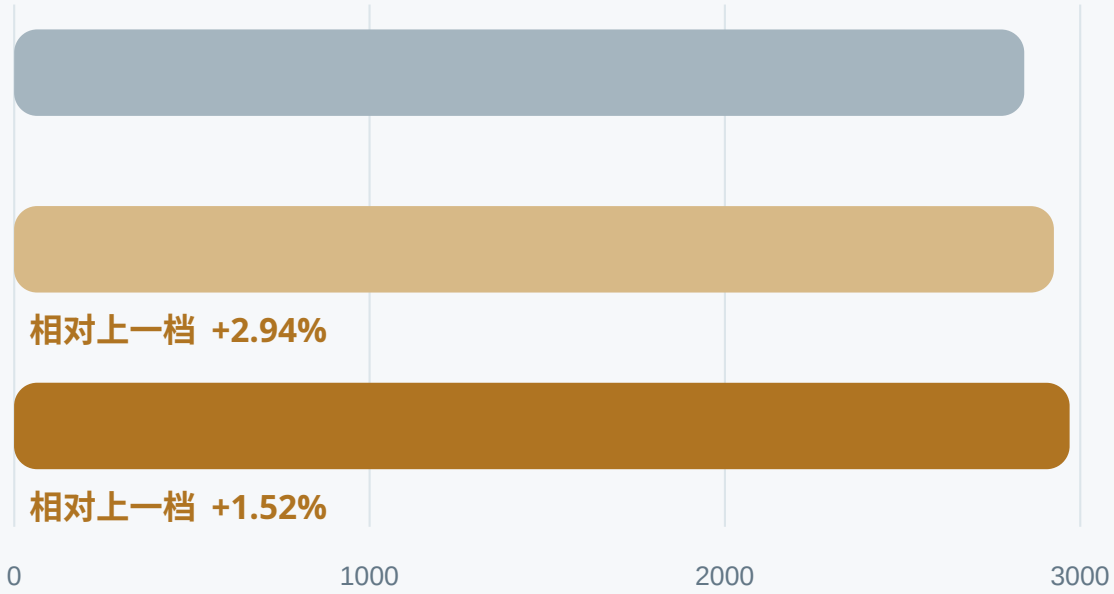
相对上一档 +2.94%

再加入

定制 EP8 通信

2,970.50

相对上一档 +1.52%



两项合计约 +4.5%；这是 Tokha233 的 modified vLLM，不是 shi3z 的消融。

来源：Tokha233 消融明细与测试方法（10/14归档）。BF16指部分稠密层执行算子；条形从零起算。

放得下权重，不代表放得下 20 个 1M 会话

DeepSeek-V4.1-Flash 的官方 checkpoint 约 510.29GB，原生采用 FP4 / FP8 存储。

官方权重制品： 510.29GB

Engram 202.76GB

放在主机内存

其余权重 307.53GB

≈ 286.41GiB，理想均分到四张 GPU

71.60 GiB / 卡

仅其余权重的四卡平均值

62.6 GiB / 卡

20 个 1M 缓存 + 1 个 prefill 临时槽

右侧缓存另加到权重之上，其他运行状态还未计入；每个 1M 会话约 2.98GiB / 卡。

依据：讲稿 三 .3 。 GB 与 GiB 分开保留；本页缓存为 shi3z 定制实现的 BF16 布局。

公司 A100 部署：固定版本的 modified vLLM

shi3z 项目用于理解优化思路与参考实现；生产部署沿用可维护的服务基础。

modified vLLM

固定经过验证的 A100 适配版本

- 请求调度
- 分页缓存
- API 接口
- 服务监控

01 只维护必要的 A100 适配补丁

把硬件适配和稳定服务接口分开。

02 在目标机器与真实 Agent 负载上测试

包含长输入、多轮工具返回和并发。

03 根据增量收益决定是否保留优化

不因演示吞吐很高就替换整个引擎。

学习工程方法；用同机、同负载测试决定上线内容。

依据：讲稿 二 .6 、三 .4 。此页保留原讲稿的部署选择。

用户感知的是完整任务耗时，不只是 token/s

以财报整理为例，生成工具调用、工具执行和结果检查都属于用户等待的一部分。



工具结果返回，进入下一轮；最后检查结果、撰写报告。

计算时间 ≈ 运算量 ÷ 有效算力

传输时间 ≈ 字节数 ÷ 有效带宽

100GB ÷ 1TB/s ≈ 0.1 秒

仅数据传输的教学算例，不包含其他阶段

来源： AI Infra Book 第 8、9、11 章与讲稿四 .1 。阶段流程示意，不代表各阶段实际耗时。

合批分摊权重，但每一路 KV 仍需分别读取

稠密模型的简化读取量：按每个输出 token 计算，而不是按整个 batch 计算。

$$\text{每个输出 token 分摊的读取量} \approx \frac{\text{权重读取量}}{\text{batch 内请求数}} + \text{该请求的 KV Cache 读取量}$$

上下文较短

权重读取占比较大，合批的摊销收益明显。

上下文较长

总吞吐可能提高，每路速度却可能下降。

Qwen： 512K → 1M

1.9×

KV 容量

3.7×

全注意力 prefill 计算

来源： AI Infra Book 与讲稿四 .2 。 1.9 倍指 KV 容量， 3.7 倍指全注意力冷 prefill 计算量。

三种手段，分别解决不同的等待来源

分块 prefill 、前缀缓存与 PD 分离不是同一个优化，也有不同的资源代价。

分块 prefill



把长输入拆成多批

批次之间处理其他请求的 decode ，减少长 prefill 对交互请求的阻塞。

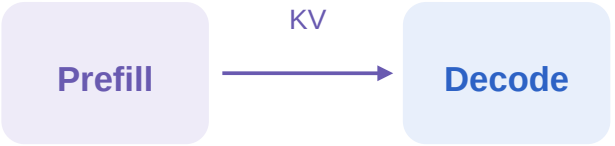
前缀缓存



避免每轮完整重算

保留已有输入的计算状态，只为新增输入做 prefill ；旧缓存仍然占用内存。

PD 分离



两组 GPU 分别调度

需要额外的模型副本与 KV Cache 传输；两组卡数应按实际负载分配。

来源： AI Infra Book 第 8 、 9 章与讲稿四 .3 。色块表示调度或状态，不表示实测运行时间。

取回已有缓存，省去长前缀的重复计算

传输估算： DeepSeek TP4 、 1M 会话；当前 Hopper 全局 KV 布局，四个 rank 合计约 7.31GB 。

读取路径	假设条件与有效带宽	载荷传输时间
CPU 内存 → GPU	7.31GB ； 四卡合计有效带宽 24GB/s	**≈ 0.30 秒**
SSD → CPU 内存 → GPU	SSD 6GB/s ， 再按 24GB/s 载入 GPU ； 串行相加	**≈ 1.52 秒**
20 个会话同时从内存载回	共 146.16GB ； 四卡合计有效带宽 24GB/s	**≈ 6.1 秒**

取回已有缓存

SSD 先读入内存，再复制到 GPU

→

只为新增输入做 prefill

处理本轮工具返回结果

→

继续生成

输出本轮回答或工具调用

前缀命中后不必重算完整长输入；排队期间还可以提前加载缓存。

仅计全局 KV 载荷；实际首 token 等待另含其他状态、排队、新增输入 prefill 与第一步生成。流程框不表示时长。

来源： handoff 六的完整输入与复算； AI Infra Book 9.5 。带宽为假设，未计首次写入。

CPU 不忙，执行环境仍可能占用大量内存

每 10 秒到达 1 项任务；每项 3 轮，每轮模型运行 60 秒、工具运行 10 秒。

每轮工具实际用量： 4 核秒 | 每个执行环境： 2GiB ，保留到任务结束

先算还没有结束的任务

21 项

3 × (60 + 10) ÷ 10

其中平均 18 项处于模型阶段

工具 CPU

1.2

核 · 平均需求

3 × 4 核秒 ÷ 10 秒

执行环境内存

42

GiB · 平均占用

21 项 × 2GiB

模型 KV 、工具 CPU 与执行环境内存，要分别预算。

依据：讲稿 四 .4 。教学假设为稳定到达、忽略排队，非真实服务性能测量。

配置选择，最终落到合格任务的时间与成本

公司： GPU + DRAM + SSD 分层； Spark： CPU / GPU 共用 128GB，重点研究统一内存 + SSD。

个人 Spark	**Qwen3.8-27B + 本地工具** 262K ≈ 18 token/s； 512K ≈ 15 token/s	共享 KV ≈ 1.81M 个人预算 6.8–8.2 万
公司 DeepSeek	**八卡 H200；比较 TP8 / 双 TP4** 缓存可容纳 20 个 1M 会话	20 路满 1M 速度 仍需对应负载测量
公司 A100	**固定版本 modified vLLM** 保留服务基础，按同机测试选择必要补丁	shi3z 项目用于学习 硬件适配与优化方法

完成一项合格任务：用了多久？花了多少？

来源：本次讲稿与 handoff。分层缓存改善会话复用，不直接改写 GPU 同时生成能力或持续 decode 速度。

单请求实测，不与四档容量估算混为一谈

单 GB10 、 TP1 ； FP8 权重 + FP8 KV ； MTP3 、 FlashInfer ； 随机 token 输入；本次实测关闭前缀缓存。

实际输入 token	输出 token	平均 TTFT / 秒	decode / token/s	口径
1,024	2,048	0.6	20.53	实测
50,000	2,048	63.1	21.01	实测
100,000	2,048	144.8	20.11	实测
300,000	2,048	658.9	17.23	实测
500,000	2,048	1,470.1	14.82	实测
999,936	**64**	**4,815.8**	**9.83***	**容量验证**

长上下文配置： YaRN factor=4 ； 主模型与 MTP 草稿上限均为 1,000,000 。

缓存命中后新增 1K 输入的 prefill 估算： 131K ≈ 2 秒 · 262K ≈ 3 秒 · 512K ≈ 5 秒 · 1M ≈ 9 秒（外推）

* 仅 64 token 的短段 decode ，不代表长输出稳态速度。四档估算均另留 8,192 token 输出空间。

来源：外部 Spark 测试汇总； Qwen 模型卡； vLLM 草稿长度问题 #58080 。配置详见本页备注。

从启动池预算换算到每会话可用上下文

公司主表按图捕获后建议预算作线性调整，再预留 10% token 余量。

公开配置	已分配 KV / 卡	实例 token 池	严格预算 / 卡
H100 TP8	12.36GiB	6.435M	10.34GiB
H200 TP4	35.16GiB	18.327M	32.49GiB
H200 TP8	62.86GiB	29.498M	60.20GiB

每会话容量 ≈ 实例 token 池 × （严格预算 ÷ 已分配 KV ） × 90% ÷ 会话数

Spark FP8 池： 1,806,907 token

BF16 近似减半：单人 512K ，或三人各 262K

Spark BF16 为同内存池近似；预留空闲 token 不会释放已分配的物理内存。

来源：公开启动池汇总与讲稿技术附注。 graph 后预算线性调整，再预留 10% ； 本表仍为 GPU 池。

公司服务的多轮性能参照：完整公开结果

平均输入约 140K–157K；前缀命中约 95%； DSpark5，平均接受长度预设为 3.51。

配置	回放会话数	总输出 / token/s	TTFT 均值 / 秒	p95 / 秒
H100 TP8	20	718.48	2.05	6.98
H200 TP4	16	614.76	0.83	3.03
H200 TP4	32	922.61	1.47	5.12
H200 TP8	16	661.15	0.65	2.14
H200 TP8	**32**	**1,052.18**	**0.85**	**2.63**

回放包含生成与等待工具两个阶段；不能据此直接推断 20 路满 1M 的每路速度。

来源： AgentX 公开回放汇总与讲稿二 .5 。沿用原测量值，保留实际输入与回放条件。

完整服务器配置与交付预算

沿用 2026-10-12 预算，假设 USD/CNY=7.00 ；点击服务器名称可查看原始报价入口。

GPU 服务器	CPU	主机内存	机内互联	预算 / 万元
8×A100 80GB SXM4 翻新	双 EPYC 7V12	1TB DDR4	HGX / NVLink	91–102
8×H100 80GB SXM5	双 EPYC 9454	1.5TB DDR5	NVSwitch	285–298
4×H200 NVL 141GB	双 Xeon 6710E 等级	1TB	四卡桥接 PCIe 服务器	161–283
8×H200 SXM 141GB	**双 EPYC 9454**	**1.5TB DDR5**	**NVSwitch**	**291–304**

四卡 H200 为组件估算， CPU 为预算等级；其余三项以完整服务器挂牌为输入。

Spark： $6,950 ；硬件交付 5.8–6.2 万元

实施：个人 4–8 人天 / 三人 8–15 人天，均按 2,500 元 / 人天

整机： 13% 税费准备 + 10% 价格预留

交付支持另列； CPU 、 RAM 、电源不重复计价；机房 / 软件 / 长期运维另计

CPU KV 池只使用扣除 Engram 、系统、工具环境和其他开销后的剩余主机内存。

来源： handoff 整机配置和报价入口；工具 CPU 节点另加 12–26 万元；具体 SKU 与交付条件采购时核对。

缓存参数不可跨实现、跨计量范围直接混用

全局 KV 、索引缓存、局部状态与临时张量的范围不同， GB 和 GiB 也不同。

当前 H100 / H200 实现

1.83 GB / 卡 / 1M 会话

仅全局 KV ； 其他状态另占显存

shi3z 的 BF16 缓存布局

2.98 GiB / 卡 / 1M 会话

全局 KV + 索引缓存

01 索引器逐头计算并累加

32 个打分头不同时保留全部分数，降低中间张量的峰值显存。

02 会话备份时去掉跨卡重复

四卡重复的全局 KV 与索引缓存，向 CPU 备份时只保留一份。

加载缓冲、临时张量、草稿模型和 CUDA Graph 地址管理，仍需按实际运行实现核对。

依据：讲稿 二 .2 、三 .3 。下方两项为 shi3z 定制实现的内存优化说明。

员工问答与通用 Agent ，共用按权限检索的资料

文件解析、权限同步与索引统一管理；检索结果携带原文出处。

NAS

文件与访问权限

→

入库与索引

解析、向量与关键词

→

权限过滤

按员工身份检索

→

引用片段

问答或 Agent 使用

Onyx Enterprise

平台、 NAS 适配与交付支持

Milvus + 自建 RAG

团队维护权限同步、索引与管理接口

内容进入重排和模型前完成权限过滤；访问原文入口时再次复核权限。

Word 、 PDF 、 PPT 、 Excel 共用入库流程；用户身份随检索请求传递。

来源：讲稿二 .1 ； 公司知识库简版回顾。 Onyx 与 Milvus 为原方案两条实施路线。

原始来源与继续阅读

六组核心入口在 PPTX 和 PDF 中均可点击；固定版本、配置条件及其他链接保留在对应页讲者备注。

01 AI Infra Book

github.com/bojieli/ai-infra-book

前缀复用、分层缓存与资源调度

02 vLLM KV 卸载指南

docs.vllm.ai/en/stable/features/kv_offloading_usage/

CPU 与文件系统层、配置与数据流

03 LMCache MP 多级存储

docs.lmcache.ai/mp/l2_storage.html

L2 存储、异步写入与预取

04 shi3z A100 项目

github.com/shi3z/deepseekv4.1-A100-custom

实现参考；固定源码见第 16 页备注

05 modified vLLM · A800 消融

github.com/Tokha233/deepseek-v4.1-flash-a100-turbo/blob/main/docs/concurrency-and-short-requests.zh-CN.md

06 Qwen / DeepSeek 模型卡

huggingface.co/Qwen/Qwen3.8-27B-FP8

huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash

正文： Spark 4–8 页｜公司 9–15 页｜ A100 16–21 页｜系统方法与总结 22–27 页；附录 28–34 页。